

DRAC HPC Resources

Allocation Request, Management, and Optimization

André Guerra

June 4th, 2026



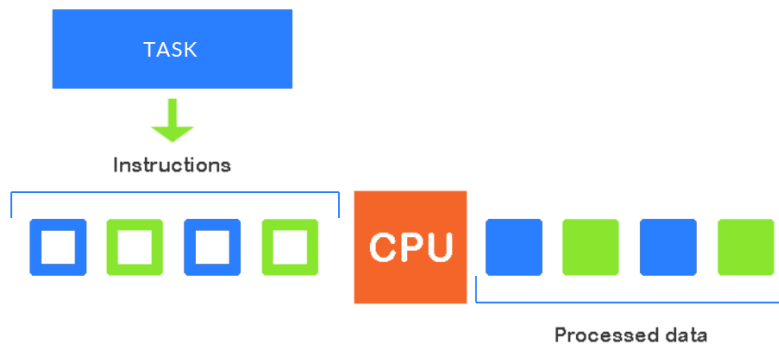
<https://github.com/DReGuerra/drac>

Running Jobs

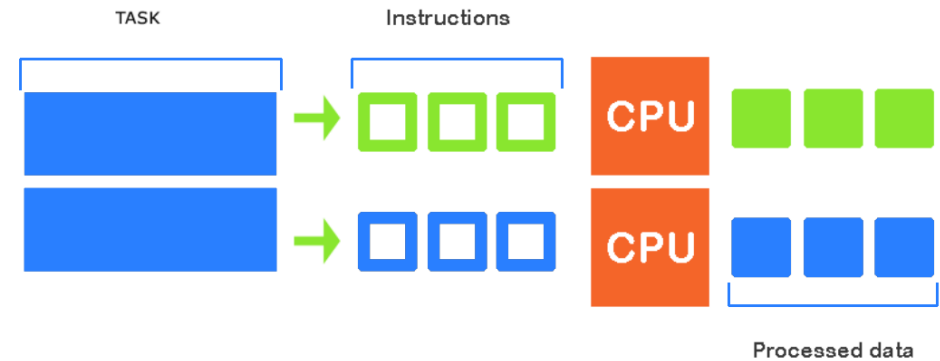
Job types, tracking, fairshare score, sbatch

Job Types

- **Serial**
 - 1 node, 1 CPU
- Sequential workloads, bash scripts, some data analysis



- **Parallel**
 - Message Passing (MPI)
 - OpenMP (multi-core threading)
 - Hybrid (MPI and OpenMP)
- HPC, advanced data analysis, image analysis



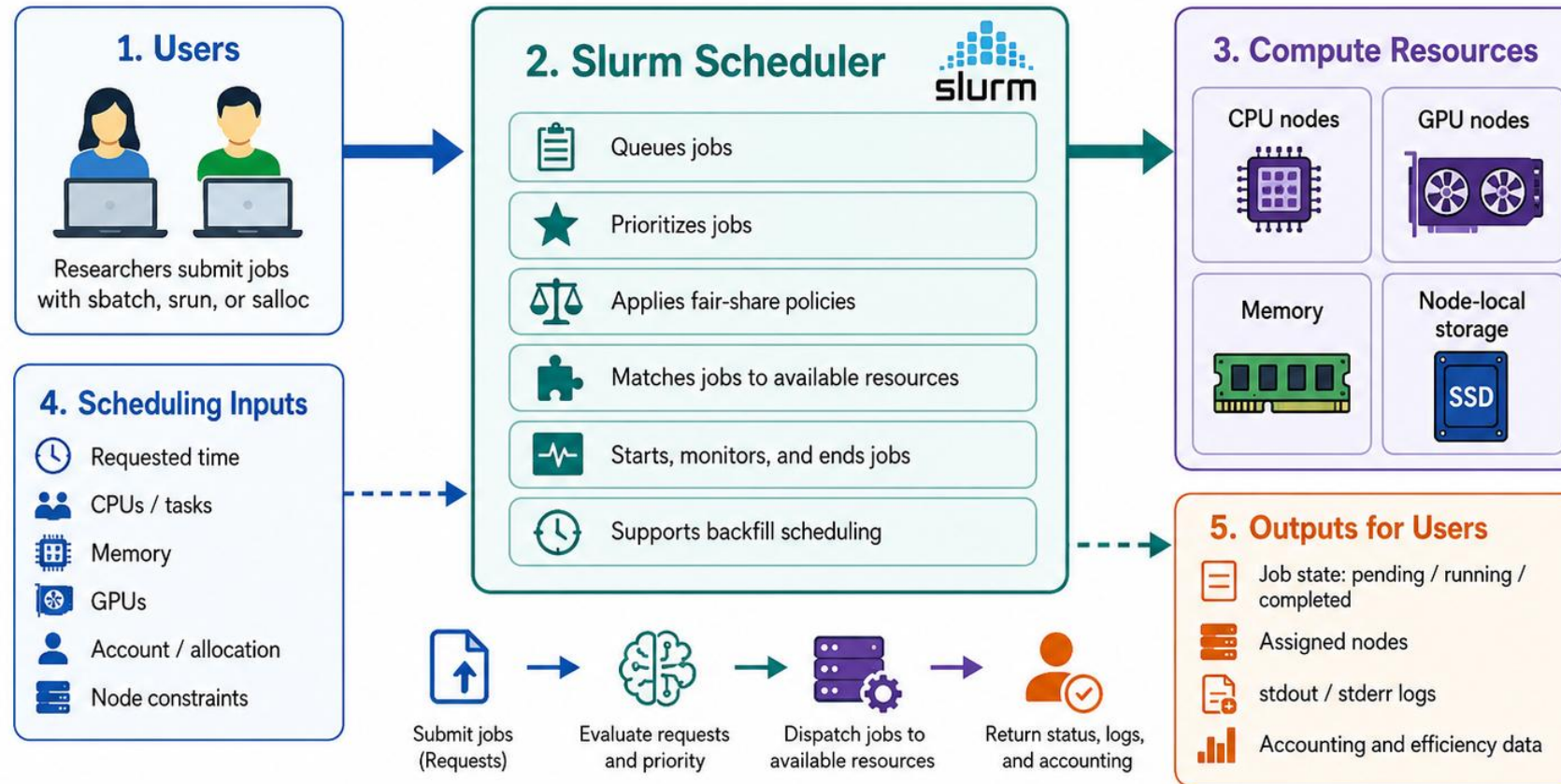
Slurm

Purpose of the Slurm Scheduler

How Slurm manages jobs and resources on an HPC cluster



Slurm is the workload manager between users and the cluster hardware.



Why Slurm matters



Prevents resource conflicts



Improves cluster utilization



Enforces shared-cluster policies



Helps users run jobs efficiently

Tracking Jobs

squeue

```
squeue | head -n 21
```

```
squeue --user=nickwzk --Format=username,name,jobid,account,numcpus,gres,minmemory,state,timelimit,timeused,reason
```

```
squeue --user=nickwzk \  
--Format=username:.10,name:.25,jobid:.10,account:.20,\  
numnodes:.6,numcpus:.6,gres:.17,minmemory:.12,state:.12,timelimit:.12,timeused:.12,reason:.10
```

```
squeue --user=$USER \  
--Format=username:.10,name:.25,jobid:.10,account:.20,\  
numnodes:.6,numcpus:.6,gres:.17,minmemory:.12,state:.12,timelimit:.12,timeused:.12,reason:.10
```

```
squeue --account=def-fzadeh_cpu \  
--Format=username:.10,name:.25,jobid:.10,account:.20,\  
numnodes:.6,numcpus:.6,gres:.17,minmemory:.12,state:.12,timelimit:.12,timeused:.12,reason:.10
```

(~/custom_aliases)

```
alias q="squeue --user=$USER \  
--Format=username:.10,name:.20,jobid:.10,account:.15,\  
numcpus:.6,gres:.17,minmemory:.12,state:.12,timelimit:.12,timeused:.12,reason:.10"
```

```
alias qq="squeue --account=def-fzadeh_cpu \  
--Format=username:.10,name:.20,jobid:.10,account:.15,\  
numcpus:.6,gres:.17,minmemory:.12,state:.12,timelimit:.12,timeused:.12,reason:.10"
```

Fairshare Score

- The *fairshare score* (FS) dictates your priority in the Slurm queue
- This depends on 2 aspects:
 - Our account's usage relative to other users of the cluster
(group component)
 - Your individual usage relative to other group members
(individual component)

Fair share score

Get the share score for all group members in the accounts specified:

```
sshare --accounts=def-kevmot_cpu,def-fzadeh_cpu -a -v --format=Account,User,NormShares,EffectvUsage,LevelFS
```

This returns:

```
Users requested:
: all
Accounts requested:
: def-kevmot_cpu
: def-fzadeh_cpu
```

Account	User	NormShares	EffectvUsage	LevelFS
def-fzadeh_cpu		0.000000	0.000001	0.006833
def-fzadeh_cpu	aguerra	0.090909	0.000000	inf
def-fzadeh_cpu	alnaba1	0.090909	0.000000	inf
def-fzadeh_cpu	ecm	0.090909	0.000000	inf
def-fzadeh_cpu	em23pff5	0.090909	0.000007	1.2521e+04
def-fzadeh_cpu	fzadeh	0.090909	0.000000	inf
def-fzadeh_cpu	legend	0.090909	0.999736	0.090933
def-fzadeh_cpu	leilap	0.090909	0.000000	inf
def-fzadeh_cpu	maxvanz	0.090909	0.000000	inf
def-fzadeh_cpu	neelcc29	0.090909	0.000256	354.502870
def-fzadeh_cpu	ohzde	0.090909	0.000000	4.6818e+05
def-fzadeh_cpu	osaur	0.090909	0.000000	inf
def-kevmot_cpu		0.000000	0.000000	inf
def-kevmot_cpu	aguerra	1.000000	0.000000	inf

```
sshare --accounts=def-kevmot_cpu,def-fzadeh_cpu -a -v --format=Account,User,NormShares,EffectvUsage,LevelFS
```

```
(~/ .custom_aliases)
```

```
alias fs="sshare --accounts=def-kevmot_cpu,def-fzadeh_cpu -a -v --format=Account,User,NormShares,EffectvUsage,LevelFS"
```

sbatch – **.out** and **.err** files

- Generate separate output and error files for troubleshooting and analysis of the jobs later (if necessary)

```
#!/bin/bash
#SBATCH --account=def-kevmot
#SBATCH --nodes=1
#SBATCH --ntasks-per-node=24
#SBATCH --cpus-per-task=8
#SBATCH --mem=0
#SBATCH --time=00-00:05:00
#SBATCH --job-name=Au22KCK16
#SBATCH --output=slurm_%A_tddft_Au22KCK16.out
#SBATCH --error=slurm_%A_tddft_Au22KCK16.err

# account name
# X server node
# X MPI ranks (cores per NUMA node)
# number of threads per rank (cores per CCD)
# memory per node
# days-hh:mm:ss
# Name of the job
# output file
# error file
```

- In the output file:
 - Generate a **job information header** in your output file to document everything
 - Generate a final **job summary** using **seff** at the end of the submission script

sbatch – job env variables

- **Job information header:**
 - Environment variables
 - Base directory
 - Modules loaded from the software stack
 - Node list (identify non-performant nodes) to later be excluded from the list

```
# helper function to print variables
print_var() {
    local name="$1"
    echo ">>> $name: ${!name:-not_set}"
}

# (...)
echo ">>> Job environment variables:"
print_var SLURM_CLUSTER_NAME
print_var SLURM_JOB_ACCOUNT
print_var SLURM_JOB_NAME
print_var SLURM_SUBMIT_DIR
print_var SLURM_JOB_ID
print_var SLURM_JOB_NODELIST
print_var SLURMD_NODENAME
print_var SLURM_MEM_PER_NODE
print_var SLURM_MEM_PER_CPU
print_var SLURM_JOB_NUM_NODES
print_var SLURM_TASKS_PER_NODE
print_var SLURM_JOB_CPUS_PER_NODE
print_var SLURM_NTASKS_PER_NODE
print_var SLURM_NTASKS
print_var SLURM_CPUS_PER_TASK
print_var SLURM_THREADS_PER_CORE
print_var SLURM_SUBMIT_HOST
print_var SLURM_CPUS_ON_NODE
print_var SLURM_ARRAY_JOB_ID
print_var SLURM_ARRAY_TASK_COUNT
print_var SLURM_ARRAY_TASK_ID

echo ">>>"
echo ">>> basedir: ${PWD##*/}"
echo ">>>"
echo ">>> Modules loaded:"
echo ">>>"
module list
echo ">>>"
```


sbatch - seff

- Job summary:

```
seff "$SLURM_JOB_ID" || true
```

```
$ seff 12345678
Job ID: 12345678
Cluster: cedar
User/Group: jsmith/jsmith
State: COMPLETED (exit code 0)
Cores: 1
CPU Utilized: 02:48:58
CPU Efficiency: 99.72% of 02:49:26 core-walltime
Job Wall-clock time: 02:49:26
Memory Utilized: 213.85 MB
Memory Efficiency: 0.17% of 125.00 GB
```

```
(...)
# Summary of the job
seff "$SLURM_JOB_ID" || true

echo ""
echo ">>> Ended task $SLURM_ARRAY_TASK_ID at: $(date) <<<"
echo "-----"

exit 0
```

Resource Optimization

Job type, hardware architecture, parallelism

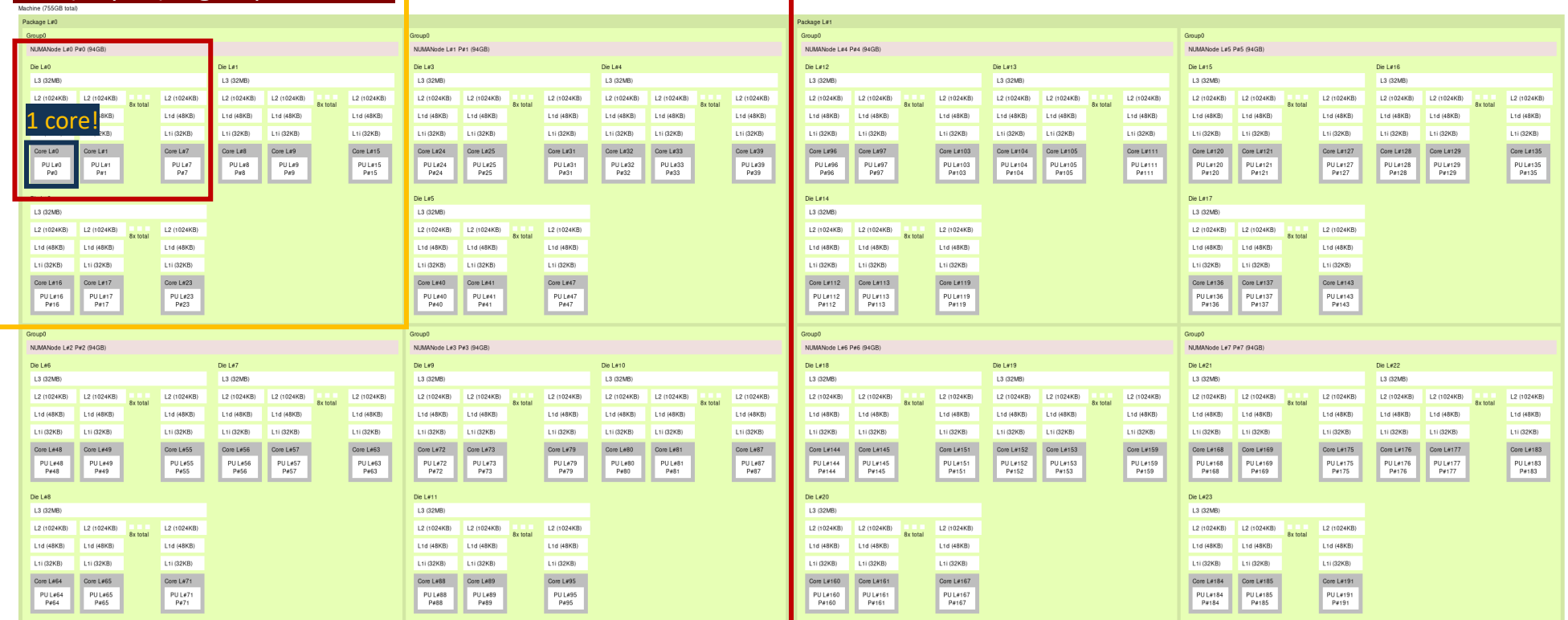
Case Study – *FIR* CPU nodes

Node (server node) – the entire compute node = $2 \times 96 = 192$ cores

Socket (Physical chip) – group of 4 NUMA nodes = $3 \times 8 \times 4 = 96$ cores

NUMA nodes – group of 3 CCDs = $3 \times 8 = 24$ cores

CCD (chiplet) – group of 8 cores



Host: s20107
Date: Fri Nov 21 09:53:17 2025

Case Study – *FIR* CPU nodes

- To optimize the requested allocation, you need to take into consideration the architecture of the specific node

CPU nodes

Architecture

Each node features 2 × AMD EPYC 9655 (Zen 5) @ 2.7 GHz processors, totaling 192 physical cores. The system is built on a chiplet-based NUMA architecture, where each chiplet (CCD) operates as a separate NUMA node. The memory and cache hierarchy is non-uniform, and performance is sensitive to data locality.

Layout

- 2 sockets, each with:
 - 96 cores
 - 4 NUMA nodes, each with:
 - 3 CCDs (chiplets), each with:
 - 8 cores
 - 32 MiB shared L3 cache
 - 3 memory channels

Each core with:

- 1 MiB L2 cache
- 32+32 KiB L1 instruction/data cache
- 12 DDR5 memory channels (shared via the I/O die)

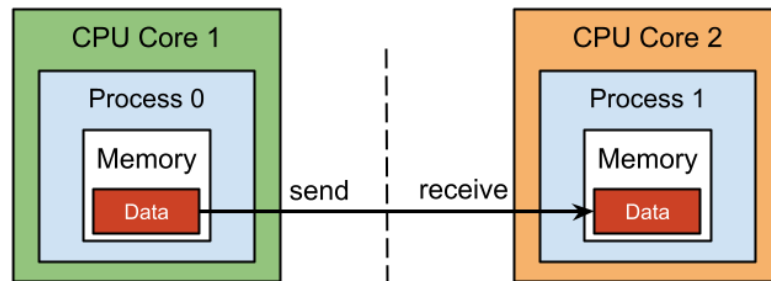
Total:

- 8 NUMA nodes per node (4 per socket × 2)
- 24 CCDs (chiplets) per node (12 per socket × 2)
- 192 cores total
- 768 MiB L3 cache total



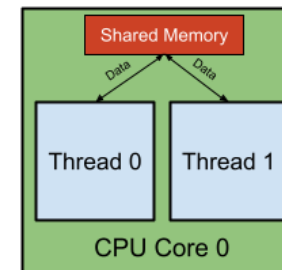
MPI

- *Message Passing Interface*
- Parallel **independent** processes (MPI ranks)
 - Memory, variables, script
- Ranks send messages to each other to coordinate



vs. OpenMP

- *Threading*
- Parallel threads with **shared**
 - Memory, variables
- Best for **single node** jobs
 - Fast access to the same memory



```
#!/bin/bash
(...)
#SBATCH --cpus-per-task=8
(...)
# set OpenMP env variable for number of threads
export OMP_NUM_THREADS=$SLURM_CPUS_PER_TASK
```

Case Study – *FIR CPU nodes*

- Why do we care about the architecture?
- *To optimize the resource usage and guide our requests to Slurm*

```
#!/bin/bash
#SBATCH --account=def-kevbots
#SBATCH --nodes=1
#SBATCH --ntasks-per-node=24
#SBATCH --cpus-per-task=8
#SBATCH --mem=0
#SBATCH --time=00-00:05:00
#SBATCH --job-name=Au22KCK16
#SBATCH --output=slurm_%A_tddft_Au22KCK16.out
#SBATCH --error=slurm_%A_tddft_Au22KCK16.err

# account name
# X server node
# X MPI ranks (cores per NUMA node)
# number of threads per rank (cores per CCD)
# memory per node
# days-hh:mm:ss
# Name of the job
# output file
# error file
```

- This uses the entire FIR node (8x24=192 cores) most efficiently – using both MPI and OpenMP threading
- Necessary when using multiple compute nodes
- *Most* computational software will be accelerated through both MPI and OpenMP threading – review their documentation to be sure, but usually safe to max out a node this way

Case Study – *FIR* CPU nodes

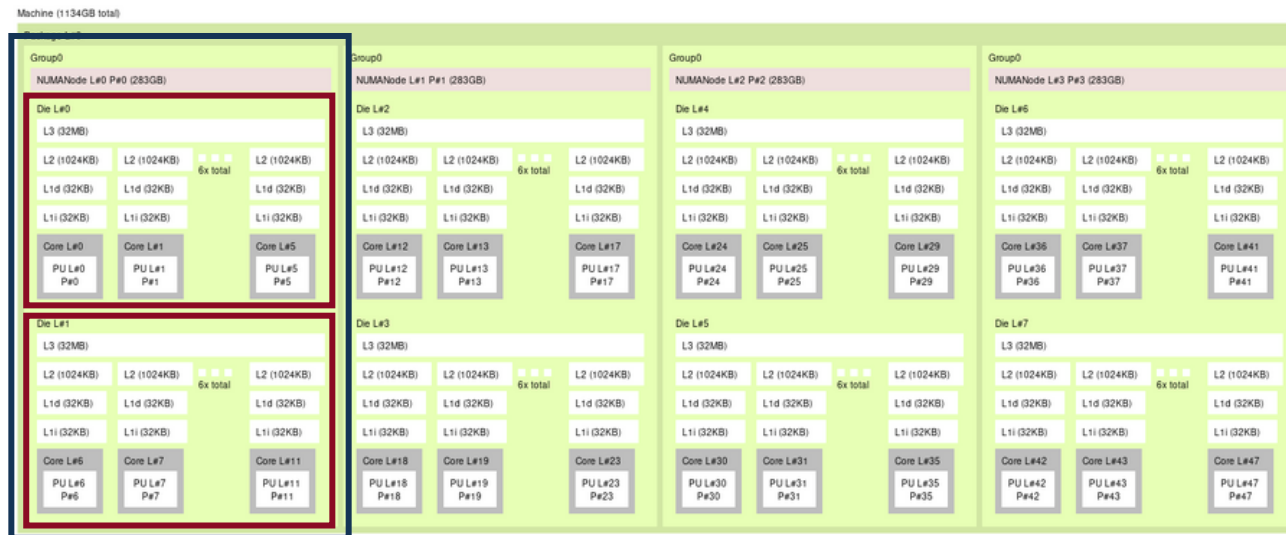
- Use subsets from this depending on the type of job you are running (MPI ranks, threaded, both)
- MPI only – bind tasks directly to cores
 - 1 MPI rank per core

```
#!/bin/bash
(...)
#SBATCH --ntasks-per-node=192          # X MPI ranks (cores per entire node)
#SBATCH --cpus-per-task=1              # number of threads per rank (cores per CCD)
```

- OpenMP threaded – bind task to CCD and thread within a CCD
 - 1 task/CCD with 8 threads (cores)
 - A purely threaded task with >8 cores may suffer from L3 memory latency – more CCDs is not always faster

```
#!/bin/bash
(...)
#SBATCH --ntasks-per-node=1          # X task (per compute node)
#SBATCH --cpus-per-task=8 # or 16,24,... # number of threads per rank (cores per CCD)
```

Case Study – FIR GPU nodes



Performance tuning recommendations

To fully utilize the architecture of the EPYC 9454 CPU and ensure optimal CPU-GPU data locality:

1. Bind threads to CCDs

Each CCD has 6 closely coupled cores sharing a 32 MiB L3 cache. To keep threads within a CCD: `#SBATCH --cpus-per-task=6`

This confines threads to one CCD, reducing cross-CCD latency and improving cache usage.

2. Match Tasks to NUMA Nodes With 4 NUMA nodes per socket (NPS=4), launch 4 tasks per node (or a multiple thereof) for best performance:

```
#SBATCH --ntasks-per-node=4 # MPI rank /NUMA node - 1x4=4
#SBATCH --cpus-per-task=12 # Threads/NUMA node - 2 CCDs (6 cores each) = 2x6=12cores
```

This keeps each task within a NUMA domain and ensures local access to memory and the GPU.

- 1 socket, configured as NPS=4:
- 4 NUMA nodes, each with
 - 2 CCDs (Core Complex Dies), each with
 - 6 cores
 - 32 MiB of shared L3 cache
 - 3 memory channels

Each core has:

- 1 MiB L2 cache
- 32 KiB L1 instruction cache
- 32 KiB L1 data cache
- 12 DDR5 memory channels (shared via the I/O die)
- 2 Nvidia H100 80GB accelerators
- The 4 node accelerators are interconnected by SXM5.

4x12=48 cores

8x6=48 cores

```
#SBATCH --n-tasks-per-node=8 # tasks (1/CCDs)
#SBATCH --cpus-per-task=6 # cores/CCD
```


Summary

- Type of jobs – **serial** or **parallel**
- Tracking jobs with **squeue** and **sshare**
- Submitting jobs with **sbatch** and specifying resources
- Take into consideration the **hardware architecture** of each cluster
- Request compute based on type of parallelization – **MPI** or **OpenMP threading**
- **GPU nodes** don't have the same architecture as the CPU nodes of the same cluster
- The **GitHub repo will be updated** as we go if more important aspects are identified

